

Gait Phase Classification Using Deep Neural Networks and Weighted Cross-Entropy Loss for Imbalanced Classes

Cesar Augusto Mendes da Silva^{*}, Felix M. Escalante[†] and Alexandre da Silva Simões[‡]

^{*,†,‡} *São Paulo State University (UNESP)
Institute of Science and Technology of Sorocaba (ICTS)
Campus of Sorocaba, Brazil*

Email: ^{*} cesar.mendes@unesp.br, [†] felix.escalante@unesp.br, [‡] alexandre.simoes@unesp.br

Abstract—We present a systematic study of gait-phase classification using fully connected deep neural networks trained with a weighted cross-entropy loss to address severe class imbalance. Our dataset comprises over 100K labeled samples from four inertial measurement units (IMUs) placed on thigh, knee, shank, and heel. We compare three network topologies of varying depth ([64, 32], [128, 64, 32], [256, 128, 64, 32] neurons), evaluating each on both 4-IMU and reduced 3-IMU inputs. Models are trained with Adam (LR = 10^{-3}), batch size 256, up to 500 epochs, and early stopping (patience = 5). We collect a rich suite of metrics, accuracy, precision, recall, F_1 , relative absolute/squared error (RAE/RSE), R^2 , training time, epochs, and confusion matrices. The deepest network (Model A) achieves 93.7 % accuracy and 0.228 RSE on 4-IMU data, dropping to 87.5 % on 3-IMUs. Mid-size Model B and compact Model C trade accuracy for speed and simplicity. Our results quantify the value of additional sensors and network capacity, guiding design of real-time embedded exoskeleton systems. Future work will explore convolutional variants for further performance gains and LSTM architectures for enhanced temporal-spatial modeling.

Keywords—gait-phase classification, deep neural networks, class imbalance, wearable exoskeleton, weighted cross-entropy, wearable robotics

I. INTRODUCTION

Gait-phase recognition is foundational for adaptive control in wearable robotics, enabling exoskeletons to modulate assistance according to the user’s locomotion subphase. Figure 1 illustrates our hardware setup: on the left, a lower-limb exoskeleton module attached to thigh, knee and shank; on the right, a simplified mechanical model comprising the motor inertia J_ω , joint inertia J_r , attachment stiffness K_a and damping B_a , and human limb impedance K_h , B_h . Accurate, real-time classification of phases such as loading response, mid-stance, terminal stance, initial swing, and terminal swing can substantially reduce interaction forces and improve comfort and safety.

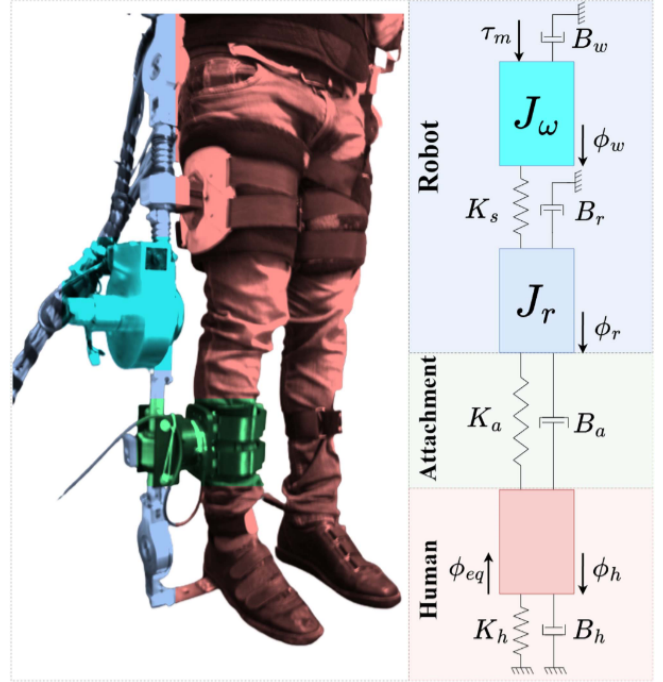


Figure 1. Wearable exoskeleton on the lower limb (left) and corresponding three-element model (right): motor inertia J_ω , joint inertia J_r , attachment stiffness/damping K_a , B_a , and human impedance K_h , B_h . [1]

Traditional model-based and threshold methods often struggle with sensor noise and inter-subject variability. Deep neural networks (DNNs) offer end-to-end learning of hierarchical features but face challenges from highly imbalanced phase distributions, some phases occur an order of magnitude more frequently than others. To mitigate this, weighted cross-entropy and class-balanced losses have been proposed in computer vision and robotics contexts.

In this work, we systematically evaluate three fully-connected DNN topologies on a large IMU gait dataset. We compute per-class weights from training labels, then train each model using Adam and early stopping. We compare

performance on both full (4-IMU) and reduced (3-IMU) inputs, reporting comprehensive metrics including accuracy, precision, recall, F_1 , RAE, RSE, R^2 , training time, and confusion matrices.

The remainder of this paper is organized as follows. Section II reviews related gait-phase recognition and class-imbalance solutions. Section III describes our dataset and preprocessing. Section IV details network architectures and training procedures. Section V presents results and discussion. Finally, Section VI concludes and outlines future directions.

II. LITERATURE REVIEW

Markovian transparency control offers a novel way to render an exoskeleton imperceptible to its wearer by modeling the gait cycle as a stochastic process. In [1], a discrete-time Markovian jump linear system (MJLS) framework is employed to switch controller dynamics in real time according to one of five gait phases. Four IMUs placed (figure 2) on the thigh ①, knee ②, shank ③, and heel ④ feed a Kalman filter estimator that identifies the current phase, enabling the controller to minimize human-robot interaction forces across transitions. Experimental validation on a series-elastic-actuator lower-limb exoskeleton demonstrated robustness to abrupt movements and a significant reduction in interaction torques, confirming the effectiveness of the Markovian approach in wearable robotics.

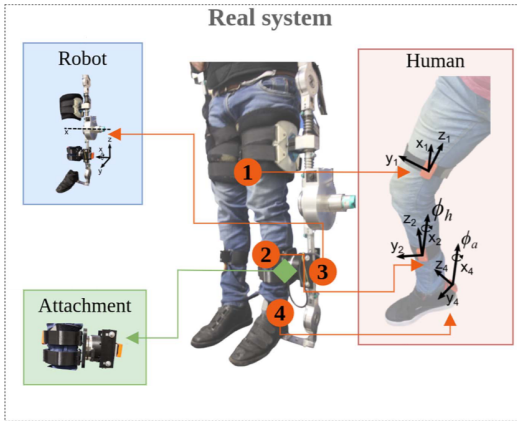


Figure 2. The transparency control hardware with IMUs locations. [1]

In [2], the authors define five Markovian operation modes that correspond to the key gait subphases (figure 3 and table I), loading response, mid stance, terminal stance, initial swing, and terminal swing, each exhibiting distinct joint-impedance characteristics. Embedding these biomechanics into a discrete-time Markov chain allows the controller to switch impedance parameters in real time: high damping is applied during loading response for impact absorption, stiffness is tuned in mid-stance to support weight transfer, torque is modulated in terminal stance in anticipation of push-off, impedance is reduced in initial swing to facilitate

foot clearance, and parameters are adjusted in terminal swing to prepare for heel strike.

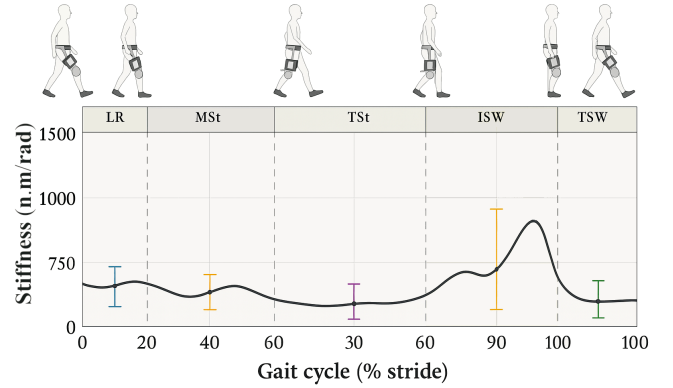


Figure 3. Estimated knee stiffness during gait cycle. [2]

Table I
ESTIMATED HUMAN GAIT PHASES. [2]

Phase	Markov mode
Loading Response (LR)	$\theta_k = 1$
Mid Stance (MSt)	$\theta_k = 2$
Terminal-Stance (TSSt)	$\theta_k = 3$
Initial Swing (ISW)	$\theta_k = 4$
Terminal Swing (TSW)	$\theta_k = 5$

Deep Neural Networks (DNNs) have demonstrated state-of-the-art performance in a wide range of classification tasks by learning hierarchical, non-linear feature representations through multiple stacked layers. A typical DNN consists of several hidden layers that apply an affine transformation followed by a non-linear activation (e.g., ReLU or tanh), culminating in a softmax output layer [3] that directly models the posterior probability of each class. Training is performed by minimizing the (weighted) cross-entropy loss between the softmax predictions and the true class labels, which corresponds to maximizing the conditional likelihood under a multinomial logistic model. Optimization is carried out end-to-end using stochastic gradient descent (or its variants) with gradients efficiently propagated through all layers via back-propagation. Theoretical analyses have shown that greater depth enables compact, highly expressive decision boundaries that would require exponentially more parameters in shallower architectures, highlighting the fundamental role of deep structures in capturing complex input-output relationships.

In [4], Phan and Yamamoto propose a weighted cross-entropy loss to address severe class imbalance, formulating the objective as

$$\mathcal{L}_{WCE} = - \sum_i w_{c_i} t_i \log P_{c_i}(x_i), \quad (1)$$

where t_i is the one-hot ground truth indicator for sample i , $P_{c_i}(x_i)$ is the softmax probability assigned to the true

class c_i , and w_{c_i} is a class-specific weight inversely proportional to class frequency. They evaluate several weighting schemes, including Balanced Cross-Entropy with fixed heuristic weights, Inverse Class Frequency, Focal Loss that modulates weights by prediction confidence, and Class-Balanced Loss based on the effective number of samples, and demonstrate substantial improvements in minority-class recall on the BDD100K benchmark [5] without significantly degrading majority-class performance.

III. PROPOSED APPROACH

Figure 4 outlines our complete gait-phase classification workflow. We begin with labeled IMU data into the five gait phases, which first passes through a data-cleaning and preprocessing stage (filtering and normalization). Next, we compute per-class weights from the training split to compensate for phase class imbalance. We then construct and configure multiple deep-neural-network architectures ($Model_n$) to identify the most effective topology. Finally, each model is trained using weighted sparse categorical cross-entropy and evaluated on held-out data, both with all four IMUs and again omitting the fourth sensor, to quantify its impact on overall performance and robustness.

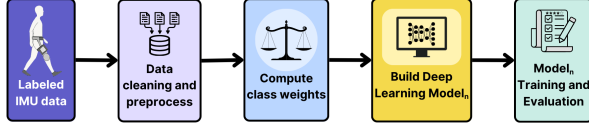


Figure 4. Gait-phase classification workflow.

IV. MATERIALS AND METHODS

A. Dataset

The IMU gait-phase dataset collected by Escalante et al. [1] comprises more than 100K samples, each labeled with one of five gait phases. Figure 5 shows the phase distribution: phase 5 alone accounts for nearly 58% of all samples, while phases 2 and 3 each comprise under 5%.

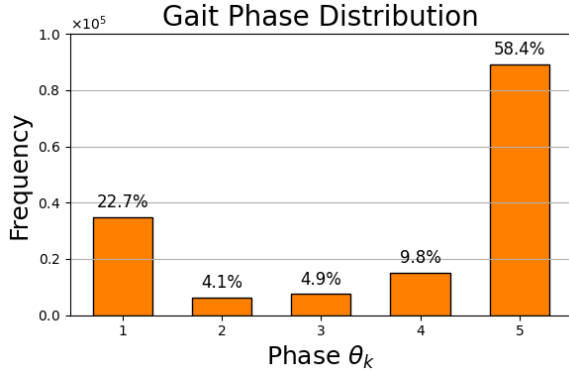


Figure 5. Distribution of the five gait phases in the dataset, shows the strong class imbalance.

B. Preprocessing

We first select the 24 raw IMU features (three gyroscope axes $G_{x_i}, G_{y_i}, G_{z_i}$ and three accelerometer axes $A_{x_i}, A_{y_i}, A_{z_i}$ for each of the four sensors) from the concatenated DataFrame. Let X denote the $N \times 24$ feature matrix and y the corresponding gait-phase labels. We then perform a stratified split using `train_test_split` (scikit-learn) as follows:

- 1) 70% of the data for training, 30% held out as a temporary set;
- 2) the temporary 30% is split 50/50 into validation and test sets, preserving class proportions.

Feature-wise standard normalization is then applied via `StandardScaler`: the scaler is fit on the training set to enforce zero mean and unit variance, and the same transformation is applied to the validation and test sets, yielding $\tilde{X}_{\text{train}}$, $\tilde{X}_{\text{val}}$, and $\tilde{X}_{\text{test}}$.

All experiments were first run using features from all four IMUs (24 channels). To assess sensor contribution, we also repeat training and evaluation omitting the fourth IMU's six features, i.e., using only 18 inputs from three sensors.

C. Class Weight Computation

After splitting and scaling, we compute per-class weights directly on the training labels to counteract the strong imbalance among the five gait phases. In Python (after the steps shown above), we use scikit-learn's `compute_class_weight` and build a dictionary mapping each label to its weight:

Algorithm 1: Computation of Weighted Class Frequencies

Input: training labels y_{train}
Output: dictionary `class_weights` mapping each class c to weight w_c

```

 $\mathcal{C} \leftarrow \text{unique}(y_{\text{train}});$ 
 $N \leftarrow |y_{\text{train}}|, K \leftarrow |\mathcal{C}|;$ 
foreach  $c \in \mathcal{C}$  do
     $N_c \leftarrow \text{count of occurrences of } c \text{ in } y_{\text{train}};$ 
     $w_c \leftarrow \frac{N}{K N_c};$ 
    class_weights[c];
     $\leftarrow w_c;$ 
end
return class_weights

```

These weights are then passed to the `class_weight` argument of `model.fit(...)` so that the sparse categorical cross-entropy loss penalizes errors on under-represented phases more heavily.

D. Network Architectures

We compare three fully-connected DNN topologies:

- **Model A:** two hidden layers with 64 and 32 neurons;

- **Model B:** three hidden layers with 128, 64 and 32 neurons;
- **Model C:** four hidden layers with 256, 128, 64 and 32 neurons.

Each hidden layer uses ReLU activation and is followed by a Dropout layer with rate 0.2. The output layer applies softmax over 5 units.

E. Training Procedure

We implement all models in TensorFlow/Keras, following the pseudocode of Algorithm 2. Models are compiled with the Adam optimizer (initial learning rate 1×10^{-3}) and trained on mini-batches of size 256 for up to 500 epochs. To counter class imbalance, we compute weighted cross-entropy using per-class weights inversely proportional to their training-set frequency. We apply an `EarlyStopping` callback monitoring validation loss with patience of 5 epochs, restoring the best-performing weights once training stalls.

Algorithm 2: Weighted Cross-Entropy Training with Early Stopping

Input: Preprocessed training ($X_{\text{train}}, y_{\text{train}}$) and validation ($X_{\text{val}}, y_{\text{val}}$) splits, class weights w_c , model topology `Model_n`

Output: Trained model `Model_n*`

```

initialize Model_n parameters;
for epoch = 1 to 500 do
    shuffle and batch ( $X_{\text{train}}, y_{\text{train}}$ ) into size-256 mini-batches;
    foreach mini-batch ( $X_b, y_b$ ) do
         $p \leftarrow \text{Model\_n}(X_b)$ ;
         $\mathcal{L} = - \sum_i w_{y_{b,i}} \log p_{i,y_{b,i}}$ ;
        update parameters via Adam step;
    end
    compute validation loss on ( $X_{\text{val}}, y_{\text{val}}$ );
    if no improvement in validation loss for 5 epochs then
        break and restore best weights;
    end
end
return Model_n*
```

F. Evaluation Metrics

Each trained model is evaluated on a held-out test set according to a comprehensive suite of metrics:

- **Overall accuracy** (fraction of correctly classified samples);
- **Precision, recall** (true-positive rate), and **F₁ score**, all computed in the weighted sense across gait-phase classes;

- **Relative absolute error (RAE)** and **relative squared error (RSE)**, measuring normalized deviation from true labels;
- **R²** coefficient of determination to assess explained variance;
- **Training time** (in seconds) and **number of epochs** until early stopping;
- **Confusion matrix**, reported both as absolute counts and row-normalized percentages for per-class insight.

All random seeds are fixed for reproducibility, and the full implementation, including data splits, model definitions, training loops, and metric collection, is available in our Google Colab notebook [6].

V. RESULTS AND DISCUSSION

Model A:

Architecture: Four hidden layers with [256, 128, 64, 32] units, dropout of 0.2 each.

4 IMUs data: For 4-IMU data, see Figures 6 and 7.

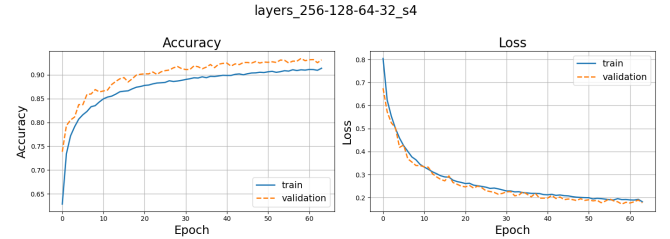


Figure 6. Model A (4 IMUs): Training and validation accuracy and loss curves.

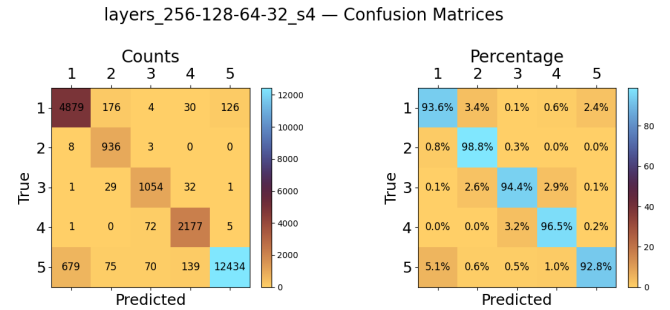


Figure 7. Model A (4 IMUs): Confusion matrix (absolute counts and normalized percentages).

3 IMUs data: For 3-IMU data, see Figures 8 and 9.

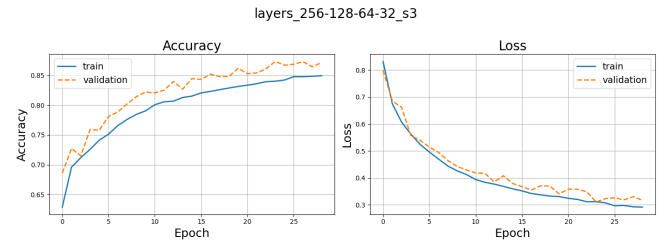


Figure 8. Model A (3 IMUs): Training and validation accuracy and loss curves.

layers_256-128-64-32_s3 — Confusion Matrices

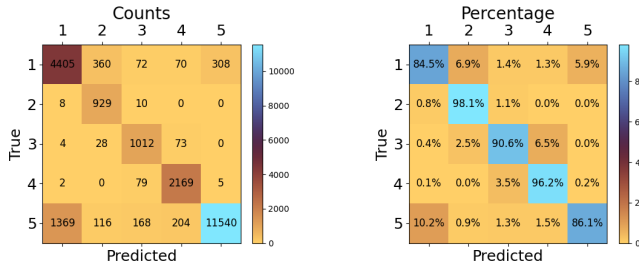


Figure 9. Model A (3 IMUs): Confusion matrix (absolute counts and normalized percentages).

Model B:

Architecture: Three hidden layers with [128, 64, 32] units, dropout of 0.2 each.

4 IMUs data: For 4-IMU data, see Figures 10 and 11.

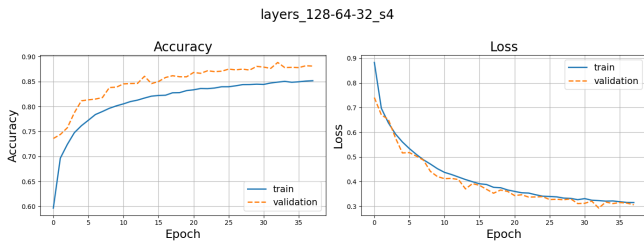


Figure 10. Model B (4 IMUs): Training and validation accuracy and loss curves.

layers_128-64-32_s4 — Confusion Matrices

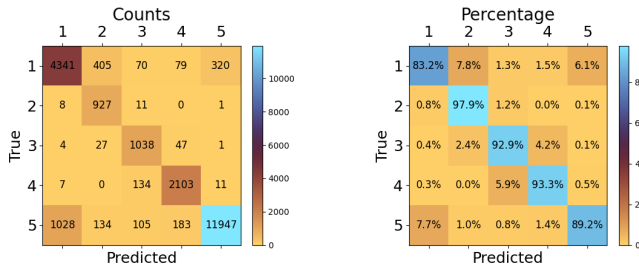


Figure 11. Model B (4 IMUs): Confusion matrix (absolute counts and normalized percentages).

3 IMUs data: For 3-IMU data, see Figures 12 and 13.

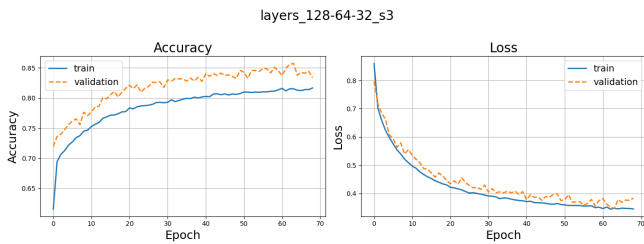


Figure 12. Model B (3 IMUs): Training and validation accuracy and loss curves.

layers_128-64-32_s3 — Confusion Matrices

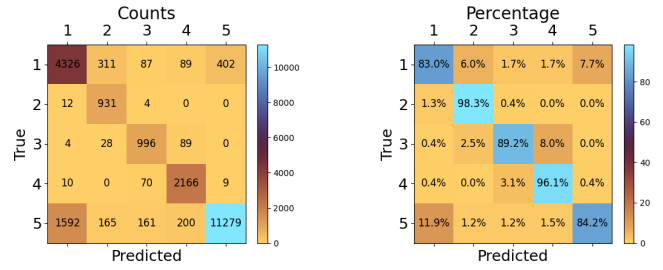


Figure 13. Model B (3 IMUs): Confusion matrix (absolute counts and normalized percentages).

Model C:

Architecture: Two hidden layers with [64, 32] units, dropout of 0.2 each.

4 IMUs data: For 4-IMU data, see Figures 14 and 15.

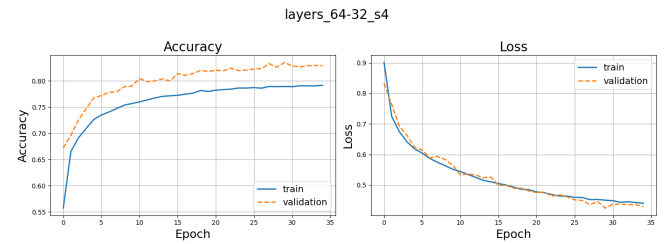


Figure 14. Model C (4 IMUs): Training and validation accuracy and loss curves.

layers_64-32_s4 — Confusion Matrices

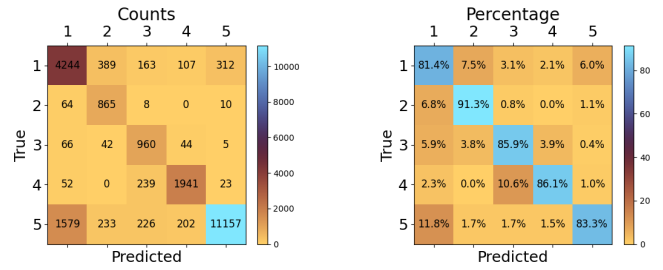


Figure 15. Model C (4 IMUs): Confusion matrix (absolute counts and normalized percentages).

3 IMUs data: For 3-IMU data, see Figures 16 and 17.

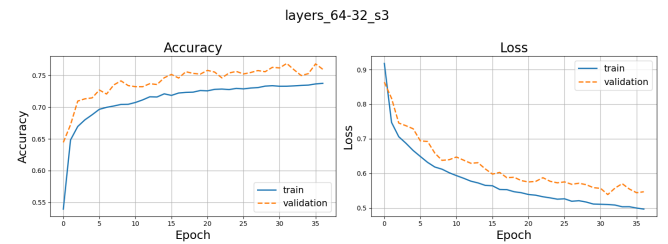


Figure 16. Model C (3 IMUs): Training and validation accuracy and loss curves.

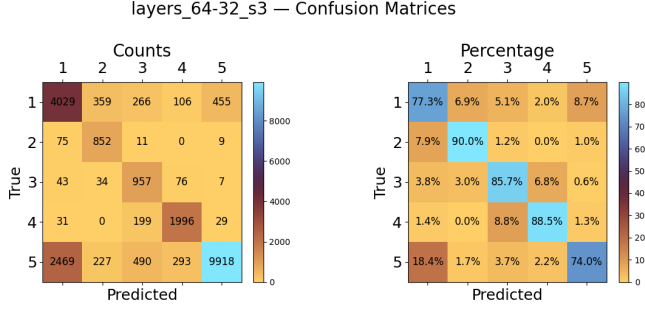


Figure 17. Model C (3 IMUs): Confusion matrix (absolute counts and normalized percentages).

Metrics:

As shown in Table II, Model A consistently outperforms the smaller architectures, achieving the highest accuracy (93.67 %) and F_1 -score (93.78 %) with four IMUs, and maintaining strong performance even when reduced to three sensors. Model B delivers moderate results (88.77 % accuracy with four IMUs), while Model C, with only two hidden layers, yields the lowest accuracy (83.59 % with four IMUs, 77.41 % with three). The relative error metrics (RAE/RSE) and R^2 values further confirm that deeper, higher-capacity models benefit more from the additional sensor; training time scales roughly with model size, peaking at 125 s for Model A (4 IMUs).

Table II

PERFORMANCE METRICS OF MODELS A–C ON 4-IMU AND 3-IMU DATA: ACCURACY, PRECISION, RECALL, F_1 , RAE, RSE, R^2 , TRAINING TIME (S), AND EPOCHS.

Model	Accuracy	Precision	Recall	F_1 Score	RAE	RSE	R^2	Training time (s)	Epochs
Model A - 4 IMUs	0.9367	0.9419	0.9367	0.9378	0.1223	0.228	0.772	125.2063	64
Model B - 4 IMUs	0.8877	0.9002	0.8877	0.8907	0.2131	0.392	0.608	80.6202	38
Model C - 4 IMUs	0.8359	0.8631	0.8359	0.8428	0.315	0.5736	0.4264	69.9836	35
Model A - 3 IMUs	0.8746	0.8902	0.8746	0.8779	0.251	0.4727	0.5273	66.9795	29
Model B - 3 IMUs	0.859	0.8765	0.859	0.8626	0.2942	0.5624	0.4376	150.1107	69
Model C - 3 IMUs	0.7741	0.8216	0.7741	0.7839	0.4563	0.8502	0.1498	80.6912	37

VI. CONCLUSIONS

Model A: The deepest architecture (four hidden layers of 256–128–64–32 units) consistently delivered the best classification performance, reaching over 93% accuracy and F_1 -score on the 4-IMU setup, and still maintaining nearly 88% accuracy when one sensor was removed. Its low RAE/RSE and high R^2 indicate robust, well-generalized decision boundaries, at the cost of longer training time (≈ 125 s) and more epochs (≈ 64).

Model B: The intermediate network (128–64–32 units) strikes a balance between capacity and efficiency. With four IMUs it achieves nearly 89% accuracy, dropping by under 3 percentage points with three sensors. Its training time (≈ 80 s) and epochs (≈ 38) are notably lower than Model A’s, making it a strong candidate when modest resource constraints are present.

Model C: The compact configuration (64–32 units) offers the fastest convergence (≈ 35 – 37 epochs, ≈ 70 – 80 s) but at the expense of reduced accuracy (84% with four

IMUs, 77% with three). Its higher RAE/RSE and lower R^2 reflect greater sensitivity to sensor reduction and simpler feature representations, acceptable for scenarios where computational footprint must be minimal.

Overall Conclusion: Across all architectures, adding the fourth IMU consistently boosts performance, with diminishing returns as model depth increases. Model A is the top choice when maximum accuracy is required, Model B offers a compelling middle ground, and Model C serves lightweight applications. In future work, we will explore convolutional network variants to further improve performance, and employ LSTM architectures to better capture local temporal–spatial correlations. We also aim to implement real-time gait-phase classification directly on an embedded exoskeleton platform.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the professors who provided invaluable guidance and support throughout this work. We also thank the University League of Autonomous Navigation and Robotics (Liga Universitária de Navegação Autônoma e Robótica, LUNAR) and the League of Neurocomputational Intelligence in Engineering (Liga de Inteligência Neurocomputacional na Engenharia, LINCE) for their collaborative assistance and resources.

REFERENCES

- [1] F. M. Escalante, L. F. d. Santos, Y. Moreno, A. A. G. Siqueira, M. H. Terra, and T. Boaventura, “Markovian transparency control of an exoskeleton robot,” *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 544–551, 2023.
- [2] F. M. Escalante, J. C. Pérez-Ibarra, J. C. Jaimes, A. A. Siqueira, and M. H. Terra, “Robust markovian impedance control applied to a modular knee-exoskeleton,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 10 141–10 147, 2020.
- [3] Y. Bengio, “Learning deep architectures for ai,” *Foundations and Trends® in Machine Learning*, vol. 2, no. 1, pp. 1–127, 2009. [Online]. Available: <http://dx.doi.org/10.1561/22000000006>
- [4] T. H. Phan and K. Yamamoto, “Resolving class imbalance in object detection with weighted cross entropy losses,” *CoRR*, vol. abs/2006.01413, 2020. [Online]. Available: <https://arxiv.org/abs/2006.01413>
- [5] F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan, and T. Darrell, “Bdd100k: A diverse driving dataset for heterogeneous multitask learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 2636–2645.
- [6] C. A. M. C. da Silva, “Gait phase classification experiments in google colab,” <https://colab.research.google.com/drive/18ltT-1bUjruiyMarr35bUtOTs6UDGun>, 2025, last accessed: 2025-06-29.